

**INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA**
São Paulo

Caderno de Problemas

1ª Fase - VIII Maratona de Programação INTERIF - 2025

28 de Junho de 2025

A PROVA TEM DURAÇÃO DE **3:00** HORAS
INÍCIO: 8h30min

APOIADORES

BIOTRONICA
<https://biotronica.tech/>

SOFTABLE
<https://softable.com.br/>

Informações gerais

Este caderno de tarefas é composto por 21 páginas (não contando a capa), numeradas de 1 a 21. Verifique se o caderno está completo.

Linguagens de programação aceitas

Nesta competição, você poderá desenvolver sua solução em qualquer uma das linguagens de programação suportadas:

- C
- C++
- C#
- Kotlin
- Java
- Python

O tempo limite que a sua solução possui para resolver um problema é o mesmo para todas as linguagens de programação suportadas. Este valor é definido no enunciado de cada problema.

Nome do código-fonte submetido

Sua solução deve ser nomeada *basename.c*, *basename.cpp*, *basename.cs*, *basename.java*, *basename.kt* ou *basename.py*, de acordo com a linguagem escolhida, onde *basename* identifica o nome do arquivo informado no enunciado de cada problema.

Sobre a entrada

- A entrada deve ser lida da entrada padrão;
- A entrada consiste em exatamente um caso de teste, que é descrito usando uma quantidade de linhas que depende do problema. O formato da entrada é como descrito em cada problema. A entrada não contém nenhum conteúdo extra;
- Todas as linhas da entrada, incluindo a última, terminam com o caractere de fim de linha (`\n`);
- Quando a entrada contém múltiplos valores separados por espaços, existe exatamente um espaço em branco entre dois valores consecutivos na mesma linha.

Sobre a saída

- A saída de seu programa deve ser escrita na saída padrão;
- Espaços em branco só devem ser colocados quando solicitado;
- Cada linha, incluindo a última, deve conter o caractere final-de-linha (`\n`).

Restrições

- Só é permitida a comunicação entre os membros de uma mesma equipe;
- Não é permitida a comunicação com o técnico (*coach*) da equipe.
- Não é permitida a utilização de dispositivos eletrônicos durante a competição.

Sistema de correção de submissões

Será utilizado o BOCA (<https://github.com/cassiopc/boca>) como sistema de submissão e de correção das soluções desenvolvidas pelos times.

Linguagens e compiladores/interpretadores

Os seguintes compiladores/interpretadores serão utilizados para compilar/interpretar os códigos-fonte das soluções submetidas para correção:

Linguagem	Compilador/Interpretador
C	GCC 11.4.0
C++	G++ 11.4.0
C#	Mono JIT 6.12.0.200
Java	Java 17.0.15
Kotlin	kotlinc-jvm 1.9.10
Python	Python 3.10.12

Os seguintes comandos serão utilizados para compilar/interpretar os códigos-fontes das soluções submetidas para correção:

Linguagem	Comando para compilação/interpretação
C	gcc -g -O2 -std=gnu11 -static -lm
C++	g++ -g -O2 -std=gnu++20 -static
C#	mcs
Java	javac
Kotlin	kotlinc -J-Xms1024m -J-Xmx1024m -J-Xss100m -include-runtime
Python	python3

Limites de recursos

Durante a compilação/interpretação e a execução de cada solução, os limites de recursos a seguir serão avaliados automaticamente pelo sistema de submissão e correção:

- Utilização de memória principal: máximo de 1GB por processo;
- Tamanho da pilha de execução: máximo de 100MB por processo;
- Tamanho do código-fonte submetido: máximo de 100KB por arquivo;
- Tamanho do arquivo de saída produzido: máximo de 1MB por arquivo.

As informações anteriores são válidas para todas as linguagens de programação utilizadas nesta prova.

Sistema de submissão

Entrada no sistema

Para utilizar o sistema, você deve realizar a autenticação utilizando as credenciais de acesso da sua equipe, disponibilizadas pela organização da competição. Não é permitida a autenticação simultânea do usuário da mesma equipe em mais de uma sessão. Caso tenha problemas com esta etapa, consulte um membro da organização da competição.

Visualização dos problemas

Uma vez autenticado no sistema, para visualizar os problemas basta acessar a aba **Problems**. Nesta aba são exibidas informações sobre todos os problemas, como a sua letra identificadora e seu título. Também é disponibilizado um arquivo, em formato PDF, contendo este caderno de problemas.

Submissão de solução

Uma vez autenticado no sistema, para realizar a submissão de um código-fonte para um determinado problema, é necessário acessar a aba **Runs**. Nesta aba, ao fim da página, deve ser informada a letra identificadora do problema (*Problem*), a linguagem de programação em que a solução foi escrita (*Language*), e o arquivo contendo o código-fonte desenvolvido. Após preencher esses dados, deverá ser pressionado o botão *Send*.

Resultado da avaliação de solução

Uma vez autenticado no sistema, é possível visualizar na aba **Runs** o resultado das submissões realizadas pela sua equipe, também chamados de **vereditos**, que podem ser um dos seguintes:

1. **YES**, indicando que a solução submetida foi aceita como correta;
2. **NO - Compilation error**, indicando que houve um erro na compilação da solução submetida;
3. **NO - Runtime error**, indicando que houve um erro na execução da solução submetida;
4. **NO - Time limit exceeded**, indicando que a execução da solução submetida atingiu o limite de tempo especificado pelo problema antes de encerrar sua execução;
5. **NO - Wrong answer**, indicando que a solução não produziu a resposta esperada;
6. **NO - Contact staff**, indicando que houve uma falha geral no processamento da solução submetida, sendo necessário entrar em contato com a organização da competição;
7. **NO - Name mismatch**, indicando que o arquivo submetido não respeita a nomenclatura especificada pelo problema. Este veredito pode acontecer apenas ao utilizar as linguagens Java e Kotlin, não acontecendo para as demais linguagens de programação suportadas nesta competição.

Penalidades

Submissões que resultem em um veredito YES, na primeira submissão da equipe para aquele problema, não serão penalizadas e acrescentarão, em uma unidade, a quantidade de problemas resolvidos pela equipe. Também será acrescido, ao tempo de resolução de problemas da equipe, o tempo decorrido de prova, em minutos, do momento da submissão da solução.

Submissões que resultem em um veredito YES após algum veredito NO, para um mesmo problema, acrescentam em uma unidade o total de problemas resolvidos pela equipe, porém, podem resultar

em penalidade para a equipe, de acordo com o veredito recebido nas submissões anteriores para aquele problema:

- Para cada submissão com um veredito dos tipos 3 (NO - Runtime error), 4 (NO - Time limit exceeded) ou 5 (NO - Wrong answer), serão acrescidos 20 minutos ao tempo de resolução dos problemas da equipe, como forma de penalidade;
- Vereditos dos tipos 2 (NO - Compilation error), 6 (NO - Contact staff) e 7 (NO - Name mismatch) não acarretam em penalidade para a equipe.

Visualização do placar

Uma vez autenticado no sistema, para visualizar o placar da competição, acesse a aba **Score**. Nela é exibida a classificação atual da competição, indicando quais equipes resolveram corretamente quais problemas, bem como a pontuação (total de acertos) e o tempo de resolução dos problemas de cada equipe (incluindo as penalidades).

Importante: o valor de pontuação considerado para a classificação final da competição é o valor apresentado na coluna **Total**, que informa o total de problemas com o veredito YES, por cada equipe, seguido do tempo de resolução destes problemas, entre parênteses, que é calculado da seguinte forma:

- Somatório do tempo decorrido de prova, em minutos, em que a solução que obteve o veredito YES, para cada problema, foi submetida;
- Somatório das penalidades para cada problema em que a equipe enviou uma solução que obteve o veredito YES, tendo submetido uma ou mais soluções anteriores, para cada um destes problemas, que obteve como resultado do processamento um veredito do tipo 3 (NO - Runtime error), 4 (NO - Time limit exceeded) ou 5 (NO - Wrong answer).

Nas colunas referentes a cada um dos problemas, é exibido o total de tentativas para aquele problema, seguido de uma barra e o tempo de resolução do mesmo. Caso a equipe não tenha submetido uma solução que obteve o veredito YES para um problema, será exibido, no lugar do tempo de resolução, o caractere -, indicando que aquele problema não será considerado no cálculo da pontuação daquele problema para aquela equipe. Caso a equipe tenha submetido uma solução que obteve o veredito YES para um problema, o tempo de resolução desse problema será calculado da seguinte forma:

- Tempo decorrido de prova, em minutos, em que a solução que obteve o veredito YES, para aquele problema, foi submetida;
- Somatório das penalidades para as submissões submetidas, para aquele problema, que obtiveram como resultado de processamento um veredito do tipo 2 (NO - Compilation Error), 3 (NO - Runtime error), 4 (NO - Time limit exceeded), 5 (NO - Wrong answer), 6 (NO - Contact Staff) ou 7 (NO - Name mismatch), submetidas antes da solução, para o mesmo problema, que obteve o veredito YES.

Nos últimos 20 minutos da prova, o placar ficará congelado. Assim, as equipes deixarão de ter acesso aos resultados das submissões realizadas por outras equipes nesse período.

Nos últimos 10 minutos, a prova ficará em modo *blind*. Isso significa que os vereditos não são enviados para a equipe quando uma submissão é realizada.

Após o término do tempo da prova, o resultado final será revelado pela organização, incluindo as pontuações e a classificação das equipes.

Esclarecimentos

Uma vez autenticado no sistema, é possível solicitar esclarecimentos sobre algum problema na aba **Clarifications**. Para solicitar um novo esclarecimento, escolha a letra identificadora do problema em *Problem* e digite um texto em *Clarification*. Caso o esclarecimento solicitado seja sobre um tema geral, e não sobre um problema específico, selecione *General* em *Problem*. Para enviar a solicitação, clique no botão *Send*.

Ainda nesta aba, são exibidas as respostas para suas solicitações de esclarecimentos, bem como esclarecimentos gerais enviados pela organização da competição.

Solicitação de impressão

Caso a sua sede tenha um serviço de impressão disponível, é possível solicitar a impressão de documentos de texto na aba **Tasks** após a autenticação no sistema. Escolha o arquivo a ser impresso e clique no botão *Send*.

Solicitação de ajuda da organização

Uma vez autenticado no sistema, caso necessite de ajuda da organização da competição, acesse a aba **Tasks**, clique no botão S.O.S e aguarde para ser atendido.

Instruções gerais para as linguagens de programação

Instruções para as linguagens C e C++

A função `main` deve retornar um valor 0 ao encerrar o programa.

Instruções para as linguagem Java e Kotlin

Não declare a classe que possui o método `main` dentro de um pacote (`package`).

Instruções para a linguagem Python

Por ser uma linguagem interpretada, erros léxicos, sintáticos ou semânticos não ocasionarão um veredito No - `Compilation error`, mas sim um veredito No - `Runtime error`.

Informações sobre os autores das questões

Este caderno possui problemas elaborados pelos docentes a seguir, ordenados em ordem alfabética:

- Carlos Frolidi - Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, *campus* Piracicaba;
- Cássio Agnaldo Onodera - Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, *campus* Birigui;
- Daniel Corrêa Lobato - Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, *campus* São José do Rio Preto;
- Érico de Souza Veriscimo - Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, *campus* São Miguel Paulista;
- Jones Mendonça de Souza - Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, *campus* Barretos;
- Tiago Alexandre Dócusse - Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, *campus* Barretos;
- Vilson Maziero - Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, *campus* Presidente Epitácio.

Problema A

Desafio de Lorde Dilon!

Arquivo: A. (c|cpp|cs|java|kt|py)

Timelimit: 1

Em Hogwarts, os estudantes de Aritmancia estão prestes a fazer uma prova prática. O professor Lorde Dilon, que manja muito de magia, lançou um desafio especial: conjurar feitiços que somem exatamente uma quantidade determinada de Pontos de Magia (PM), utilizando apenas pedras mágicas especiais que possuem poderes fixos.

Cada pedra mágica tem um valor de poder que corresponde a uma potência de dois positiva (como 1, 2, 4, 16, 128, etc.), e cada uma das pedras disponíveis pode ser usada quantas vezes o aluno quiser. Seu objetivo é descobrir qual o menor número de pedras mágicas que um aluno pode usar para atingir exatamente a quantidade de Pontos de Magia desejada.

Se não houver como conjurar exatamente a quantidade necessária de PM com as pedras fornecidas, o aluno está condenado a repetir o semestre.

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 10$), representando a quantidade de tipos diferentes de pedras mágicas fornecidas pelo professor.

As N linhas seguintes contêm, cada uma, um número inteiro P ($1 \leq P \leq 10000$), representando o poder (valor da potência de dois) de cada tipo de pedra mágica.

A última linha contém um número inteiro V ($0 \leq V \leq 10^9$), representando a quantidade exata de Pontos de Magia que o feitiço precisa consumir.

Saída

Um único número inteiro representando a menor quantidade de pedras mágicas necessárias para conjurar o feitiço com exatamente V Pontos de Magia.

Se não for possível conjurar exatamente esse valor, imprima: frustraka.

Exemplos

Entrada	Saída
5 8 1 2 4 16 21	3

Entrada	Saída
2	6
4	
128	
24	

Entrada	Saída
2	frustraka
4	
128	
15	

Problema B

Dança das Letras

Arquivo: B.(c|cpp|cs|java|kt|py)

Timelimit: 1

Durante suas experiências com cadeias de DNA muito peculiares, a pesquisadora Ana se deparou com uma sequência composta apenas pelas letras 'A' e 'B'. Curiosa, ela decidiu investigar qual o maior trecho da sequência onde as letras se alternam sem repetir duas iguais seguidas. Ela chamou esse tipo de trecho de "dança das letras".

Sua missão é ajudá-la a escrever um programa que, dada uma *string* com letras 'A' e 'B', informe o tamanho da maior *substring* alternante contínua. Sabendo-se que uma *substring* alternante é uma sequência contínua de caracteres dentro da *string* original, onde os caracteres se alternam estritamente entre 'A' e 'B'. Em outras palavras, dois caracteres consecutivos nunca podem ser iguais. Exemplos: "AB", "BA", "ABA", "BAB", "ABAB", "BABABA".

Entrada

Uma única *string* S , com no máximo 10^5 caracteres. A *string* é composta apenas pelos caracteres 'A' e 'B'.

Saída

Um único número inteiro representando o comprimento da maior *substring* alternante contínua da *string*.

Exemplos

Entrada ABABBA	Saída 4
Entrada AAAAAA	Saída 1
Entrada BABABABABA	Saída 10

Problema C

A Conspiração dos Professores do IFSP

Arquivo: C.(c|cpp|cs|java|kt|py)

Timelimit: 1

No Instituto Federal de Educação, um fenômeno curioso ocorreu. Os professores de diferentes áreas – Matemática, Informática, Física e Administração – receberam uma misteriosa convocação para uma grande reunião interdepartamental. O objetivo? Criar um novo modelo de ensino que revolucionará a forma como algoritmos e estruturas de dados são ensinados no Brasil.

Mas há um problema... Cada professor tem uma carga de energia limitada (você sabe como são as reuniões). E a energia total necessária para organizar a reunião depende de como os grupos de professores são divididos e reunidos novamente.

Reunir todos os N professores de uma vez só é impossível. O segredo está em dividir esse grupo em dois subconjuntos e realizar sub-reuniões em pares, reduzindo o custo total. Mas há várias formas de fazer isso, e os organizadores precisam encontrar a forma que consome menos energia.

Regras da Energia:

A energia para reunir N professores é calculada da seguinte forma: $energia(N) = \min(energia(i) * energia(N - i))$ para todo i em 1 até $N - 1$

Com os seguintes valores iniciais fixos:

- $energia(1) = 2 \rightarrow$ Convencer um único professor a participar de algo novo exige 2 unidades de energia.
- $energia(2) = 3 \rightarrow$ Reunir duas mentes críticas já dá um bom trabalho.

Entrada

Um número inteiro N ($3 \leq N \leq 70$), representando o número total de professores a serem reunidos.

Saída

Um único inteiro: o menor custo de energia para reunir todos os N professores.

Exemplos

Entrada 3	Saída 6
Entrada 10	Saída 243

Problema D

Competição de Força Bruta

Arquivo: D. (c|cpp|cs|java|kt|py)

Timelimit: 1

Em uma Competição de Força Bruta (CFB), competidores são avaliados em três tentativas de levantamento de peso. Os pesos levantados são registrados como números inteiros. Para determinar o vencedor de cada rodada, as seguintes regras são aplicadas:

- Um competidor que levanta o mesmo peso três vezes ("triplo") supera um competidor que levanta o mesmo peso duas vezes ("duplo");
- Em um confronto entre dois competidores com "triplo", o competidor com o maior peso levantado vence;
- Em um confronto entre dois competidores com "duplo", o competidor com o maior peso levantado vence;
- Se não houver um vencedor claro (ou seja, nenhum competidor tem um "triplo" ou "duplo"), o resultado da rodada é empate.

Um competidor já realizou duas tentativas de levantamento, e os pesos levantados foram registrados. Sua tarefa é determinar qual peso ele deve levantar em sua terceira tentativa para maximizar suas chances de vencer a rodada.

Entrada

A entrada consiste em uma única linha contendo dois números inteiros, A e B ($1 \leq A, B \leq 1000$), representando os pesos levantados nas duas primeiras tentativas do competidor.

Saída

O programa deve produzir uma única linha com um número inteiro, indicando o peso que o competidor deve levantar em sua terceira tentativa para maximizar suas chances de vencer a rodada.

Exemplos

Entrada 125 112	Saída 125
Entrada 99 134	Saída 134
Entrada 57 57	Saída 57

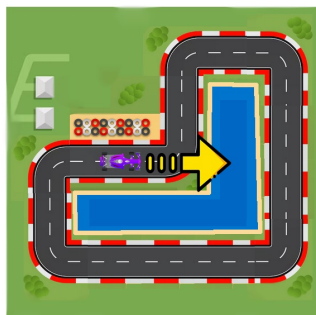
Problema E

Grande prêmio de Fórmula 2

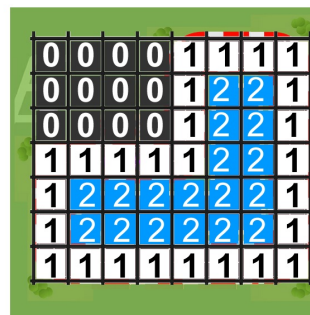
Arquivo: E.(c|cpp|cs|java|kt|py)

Timelimit: 1

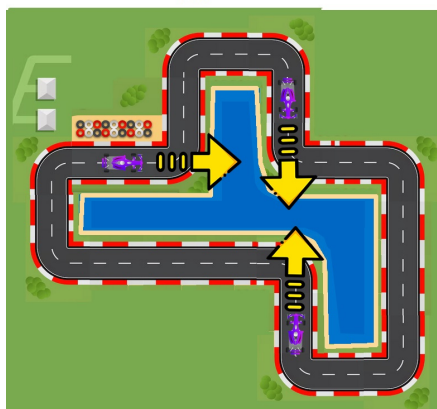
Depois de muito esforço e dedicação, Bryan alcançou a vitória na última temporada da Fórmula 3. Seu irmão e empresário, Luiggy, tem perdido noites de sono desde que soube que Bryan aceitou o convite de uma equipe para competir na Fórmula 2. Luiggy sabe que todos os circuitos da Fórmula 2 foram construídos em torno de pequenas represas/lagos, o que é um fator preocupante para o seu irmão. Aos 8 anos, Bryan competia como piloto de *kart* e sofreu um acidente ao cair na represa da onça no circuito de Tamandaré, em Rio Branco. Para evitar traumas do passado, Luiggy planeja analisar cada circuito e destacar os pontos mais perigosos para Bryan, garantindo um treinamento adequado e um desempenho satisfatório na busca pelo campeonato da Fórmula 2. No entanto, são muitos mapas de pistas para analisar, e ele precisa da sua ajuda para identificar esses pontos críticos. Você poderia ajudá-lo?



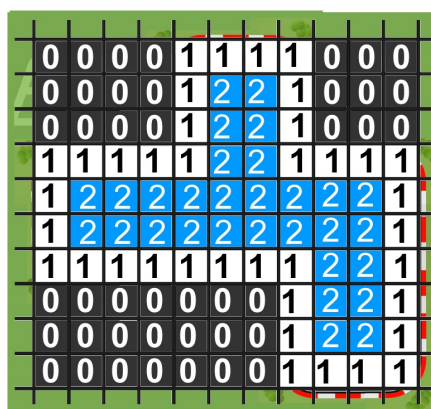
(a)



(b)



(c)



(d)

Que bom que você aceitou! Luiggy separou dois mapas do circuito, eles estão ilustrados na Figura (a) e (c). Observe que os carros circulam sempre no sentido horário e, analisando o circuito ilustrado em (a), pode-se concluir que há somente uma curva em que Bryan precisaria tomar mais cuidado. Já no circuito apresentado em (c), há três pontos de perigo. Para te ajudar, Luiggy pediu para a Federação Internacional do Automóvel uma representação numérica dos circuitos, assim como ilustrados na Figura (b) e (d). Os pontos nas matrizes (b) e (d), representados com o inteiro 1, indicam o trecho do circuito. Já os valores representados com o inteiro 2 indicam a região da represa/lago. Os demais valores representados com 0 representam regiões com gramado no circuito.

Entrada

A primeira linha possui dois inteiros N e M ($3 \leq N, M \leq 500$), que representam as dimensões da matriz. Cada uma das N linhas seguintes contém M inteiros ($0 \leq M_{ij} \leq 2$), os elementos da matriz separados por um espaço em branco, representando o circuito da corrida.

Saída

Imprima a quantidade de curvas que Bryan deverá ter cautela para não cair na água da represa/lago.

Exemplos

Entrada	Saída
7 8 0 0 0 0 1 1 1 1 0 0 0 0 1 2 2 1 0 0 0 0 1 2 2 1 1 1 1 1 1 2 2 1 1 2 2 2 2 2 2 1 1 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1	1

Entrada	Saída
10 10 0 0 0 1 1 1 1 0 0 0 0 0 0 1 2 2 1 0 0 0 0 0 0 1 2 2 1 0 0 0 1 1 1 1 2 2 1 1 1 1 1 2 2 2 2 2 2 2 2 1 1 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 2 1 0 0 0 0 0 0 0 1 2 1 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0	3

Problema F

Contando palavras

Arquivo: F.(c|cpp|cs|java|kt|py)

Timelimit: 1

Rafael é um menino muito inteligente que adora ler livros de historinhas. Além das histórias, é claro, o que ele mais gosta nos livros são as palavras, e como elas aparecem dentro de cada livro, pois a composição das palavras, e como os autores utilizam essas palavras, é algo que o fascina e o deixa animado, a cada vez que sua mãe escolhe um livro novo para ler com ele, antes de dormir.

Ele está crescendo, no entanto, e outra área dos estudos que o fascina é a matemática. Não só realizar contas básicas como adição, subtração, multiplicação e divisão, mas também os cálculos estatísticos. E para juntar essas duas paixões de Rafael, ele teve uma ideia: contar as palavras nos seus livros de historinha favoritos. Ele começou a fazer esse processo de forma manual. No entanto, percebeu que este é um processo monótono e extremamente lento, e se desinteressou da atividade.

Rafael, no entanto, ficou super animado quando seu pai disse que a maratona de programação InterIF será realizada, pois sabe que há ótimos programadores participando desta competição e que, com certeza, vários grupos conseguirão automatizar este processo, permitindo assim que Rafael possa visualizar os dados dos seus livros favoritos. Você consegue ajudá-lo com este processo?

Entrada

A entrada do seu programa deve ser um texto sem formatação, contendo várias frases. Garante-se que o texto de entrada é formado por ao menos uma palavra e não há linhas em branco. Sinais de pontuação (como vírgula, ponto-e-vírgula, ponto final, dois pontos, aspas, ponto de exclamação e ponto de interrogação) não devem ser considerados no texto. Não há palavras com acentos ou caracteres especiais nas entradas.

Saída

O seu programa deve exibir, em ordem decrescente de aparição, a quantidade de ocorrências de palavras em uma linha, seguida do caractere “:”, de um espaço em branco e de todas as palavras, ordenadas alfabeticamente e separadas por um espaço em branco, que tenham aquela quantidade de ocorrências no texto. Todas as palavras devem ser exibidas com letras minúsculas.

Exemplos

Entrada	Saída
Quem tem boca vai a Roma. Que tem boca via Roma. O filosofo se perguntava: Quem tem boca "vai a" ou "via" Roma?	3: boca roma tem 2: a quem vai via 1: filosofo o ou perguntava que se

Problema G

Alfabetizando as crianças

Arquivo: G.(c|cpp|cs|java|kt|py)

Timelimit: 1

O processo de alfabetização começa pelo reconhecimento das letras, depois avança com os sons silábicos e pequenas palavras. A professora Valdinéia está com dificuldades em ensinar as crianças da sua turma a reconhecer a primeira letra de uma palavra. Para resolver este problema, ela teve a ideia de criar um aplicativo que seja capaz de auxiliar as crianças com essa atividade.

Como a professora Valdinéia não sabe programar, ela resolveu lhe contratar para desenvolver este aplicativo. Sua tarefa é escrever um programa que receba uma palavra e exiba a primeira letra dela. É garantido que a entrada esteja em letras maiúsculas.



Entrada

A entrada do programa é composta por uma única palavra, que deve possuir de 2 até 100 caracteres.

Saída

Imprima a primeira letra da palavra de entrada.

Exemplos

Entrada ABELHA	Saída A
Entrada BOLA	Saída B

Problema H

Uma casinha no meio do nada... Ou não.

Arquivo: H.(c|cpp|cs|java|kt|py)

Timelimit: 1

Em um vasto e silencioso *grid*, existia uma casinha solitária. Ou pelo menos era o que ela pensava... Dado um *grid* de L linhas por C colunas, a posição de uma casinha em (x_0, y_0) e as posições de outras H casas espalhadas pelo *grid*, sua tarefa é determinar se a casinha está realmente "no meio do nada" ou se está "no meio de todos". Uma casa é considerada no meio do nada se a distância euclidiana até todas as outras H casas for estritamente maior que D . Caso exista pelo menos uma casa a uma distância menor ou igual a D , então ela está no meio de todos.

Entrada

A entrada é composta por várias linhas. A primeira linha contém cinco inteiros: L , C , x_0 , y_0 e D — representando, respectivamente:

- o número de linhas do grid ($1 < L \leq 10^5$);
- o número de colunas do grid ($1 < C \leq 10^5$);
- a coordenada x_0 da casinha principal ($0 < x_0 \leq L$);
- a coordenada y_0 da casinha principal ($0 < y_0 \leq C$), e;
- a distância máxima D para que a casinha esteja "no meio do nada" ($0 \leq D \leq 15000$).

A segunda linha contém um inteiro H ($0 \leq H \leq 10^5$) representando o número de outras casas no *grid*.

As H linhas seguintes contém duas coordenadas inteiras X_i e Y_i ($0 \leq X_i \leq L$ e $0 \leq Y_i \leq C$) representando as posições das demais casas.

Saída

Seu programa deve imprimir uma única linha:

- "Uma casinha no meio do nada" caso a casa em (x_0, y_0) esteja a mais de D unidades de todas as outras casas, ou;
- "Uma casinha no meio de todas" caso exista alguma casa a uma distância menor ou igual a D .

Exemplos

Entrada	Saída
10 10 3 3 2 3 1 1 5 5 8 8	Uma casinha no meio do nada

Entrada	Saída
10 10 0 0 5 1 3 4	Uma casinha no meio de todas

Problema I

Conversando em segredo

Arquivo: I.(c|cpp|cs|java|kt|py)

Timelimit: 1

Marina e Rafael são duas crianças que aprontam muita confusão, e também gostam de conversar e trocar ideias sobre as bagunças que irão fazer em casa. No entanto, seus pais vêm escutando o que eles estão planejando atualmente, o que tem atrapalhado seus planos de diversão. Claro que essa tranquilidade para os pais deles não irá durar muito tempo, visto que eles já possuem uma ideia de como contornar esta situação: conversando em código.

Eles ganharam do tio deles um presente, que possui uma roleta contendo todas as letras do alfabeto, e pensaram na seguinte ideia: vamos escrever nossas mensagens em texto, porém, ao invés de escrever o texto com os planos das brincadeiras, elas vão substituir as letras por outras letras. Assim, os pais de Marina e Rafael não vão entender o que está escrito no texto, mas como eles sabem como substituir as letras, vão conseguir entender o que o outro quis dizer e, assim, aprontar sem que seus pais saibam o que está por vir.

Por sugestão de Marina, eles combinaram de substituir as letras da seguinte forma: para a primeira letra da palavra, roda-se a roleta no sentido horário; já para a segunda letra, roda-se a roleta no sentido anti-horário; e assim continua o processo para cada letra, horário, anti-horário, horário, anti-horário, até que não haja mais letras a serem analisadas. Marina ficou com uma dúvida, no entanto: quanto rodar a roleta para cada letra? Pois se a quantidade for a mesma para todas as letras, ficará fácil para seus pais descobrirem seus planos. Rafael, no entanto, teve uma ideia: vamos rodar a roleta por N casas, de forma que, para cada letra, um valor específico de N seja utilizado. Mas ao invés de guardar os números, que só poderiam ser de 1 a 9, eles vão guardar letras, de acordo com sua ordem no alfabeto, de forma a armazenar valores de 1 a 26, sendo $A = 1$, $B = 2$, $C = 3$, e assim sucessivamente, formando uma senha a ser utilizada com a mensagem escolhida.



A ideia deu certo e as crianças estão conseguindo se comunicar. Porém, elas acharam esse processo tedioso, pois gastam mais tempo calculando a mensagem secreta e depois recuperando a mensagem original que, de fato, se divertindo. Por isso, elas precisam da sua ajuda: você consegue criar um algoritmo capaz de fazer o processo e permitir que elas possam brincar por mais tempo?

Entrada

A entrada do programa é formada por diversas linhas. Cada linha inicia com o número 1, caso a mensagem original deva ser transformada para a sua versão secreta, ou 2, caso a mensagem secreta deva ser retornada para a versão original. Após o número, em cada linha seguem um espaço em branco e dois conjuntos de caracteres, separados por um espaço em branco, sendo o primeiro conjunto a mensagem e o segundo a senha a ser utilizada. A entrada é encerrada quando uma linha contendo apenas o valor 0 (zero) é lido. A entrada deve sempre conter mensagens e senhas em caixa baixa e de mesmo comprimento.

Saída

Exibir em cada linha o valor referente à operação informada sobre a mensagem informada com a senha informada. A saída deve sempre ser exibida em caixa baixa.

Exemplos

Entrada	Saída
1 institutofederal chaveaseutilizar	fvrpdubytzvpvrzd
2 ozysnssvpt outrachave	desaopaulo
0	

Problema J

Cofre Mágico

Arquivo: J.(c|cpp|cs|java|kt|py)

Timelimit: 1

No antigo templo de Numéria, existe um cofre mágico que só pode ser aberto com um número especial. Para isso, o guardião precisa encontrar, dentro de uma sequência de números, um único número que tenha a maior quantidade de divisores positivos. Caso haja empate, o guardião deve escolher o menor número dentre os empatados.

Você foi convocado para escrever um programa que determine esse número especial!

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 10^5$), o tamanho da sequência. A segunda linha contém N inteiros $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^6$), os números da sequência.

Se no lugar do número inteiro for digitado o valor 0, o programa deve ser finalizado.

Saída

Imprima o número da sequência que possui a maior quantidade de divisores positivos. Em caso de empate, imprima o menor número.

Exemplos

Entrada 5 10 15 12 8 9	Saída 12
Entrada 3 6 28 36	Saída 36
Entrada 4 7 11 13 17	Saída 7